Reinforcement Learning with Grid World

Aaron Cuthinho & Sai Mukkundan Ramamoorthy

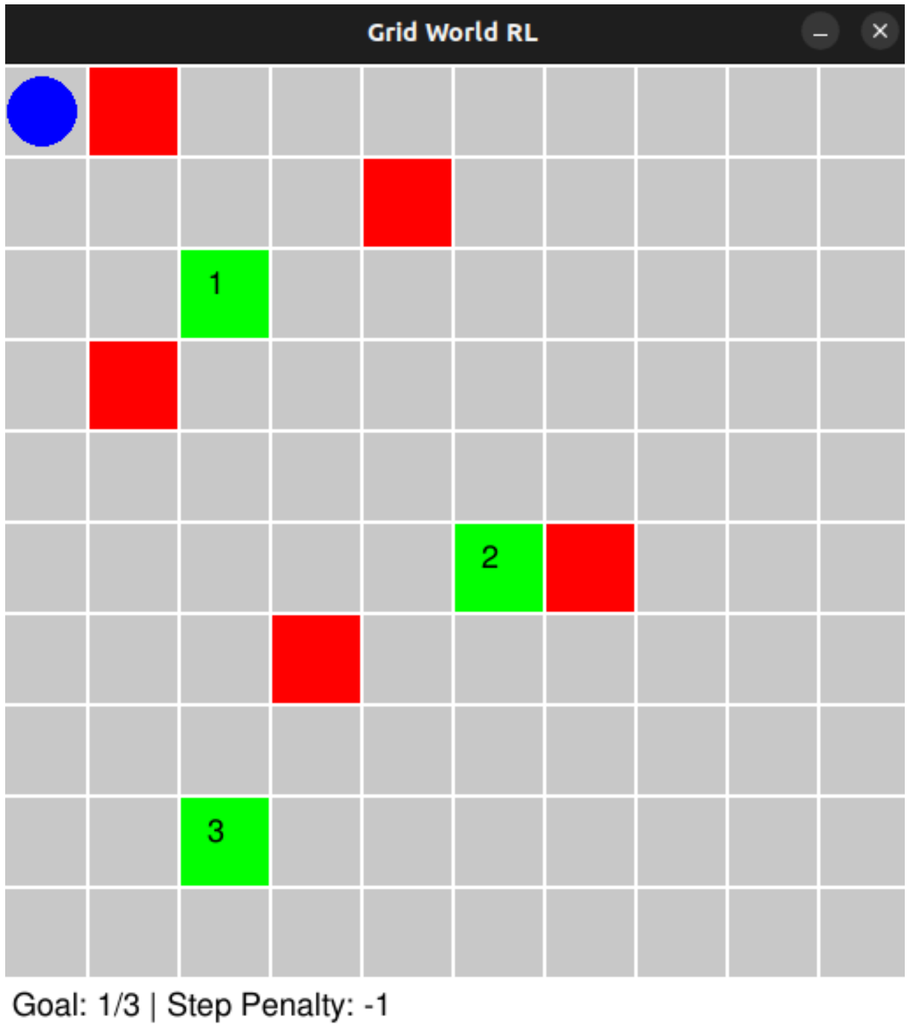
ML Course WS-2025/26

Jan 21, 2026

Project Overview

Goal:

- Training an RL agent to navigate a gridworld to reach a sequence of goals while avoiding dynamic obstacles.
- Comparing the RL agent performance based on the type of algorithm used. (Traditional vs Deep RL Algorithm)



Grid World (1/2)

The 2D grid world is made using PyGame. PPO training was done using skrl.

How it works :

- Grid Size: 10×10 cells
- Goal Sequence: 4 sequential green goals to reach in order
- Agent Actions (4 directions):
  - Move Up | Move Right | Move Down | Move Left
- Obstacle Collision: Agent bounces back to previous position

Grid World (2/2)

Reward Structure :

Positive Rewards

- +100 for reaching a goal
- +50 bonus for completing entire sequence

Negative Rewards

- -1 for each step taken (efficiency penalty)
- -10 for hitting an obstacle
- -10 if an obstacle moves onto agent

Q Learning (1/2)

- **Algorithm Type**: Value-based, Model-free Reinforcement Learning
- **Goal**: Learn Q-values: the expected future reward for taking action a in state s
- **No Environment Model**: Agent learns purely from trial and error, not from knowing how the world works
- **Policy**: Implicit—always choose the action with highest Q-value (greedy)
- **Q-Table**: Dictionary mapping (state, action) pairs to learned Q-values
- **Epsilon-Greedy Strategy**: Epsilon-greedy balances exploration and exploitation by choosing a random action with probability ε, otherwise selecting the action with the highest Q-value.

Q Learning (2/2) - Bellman Equation

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R(s,a) + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

Where:

- **Q(s,a)**: The Q-value for taking action a in state s.
- **α**: The learning rate, which determines how much new information overrides the old Q-value.
- **R(s,a)**: The immediate reward received after taking action a in state s.
- **γ**: The discount factor, which weighs future rewards relative to immediate rewards.
- **$\max_{a'} Q(s',a')$**: The maximum Q-value for the next state s′, across all possible actions a′.

This equation updates the Q-value based on the immediate reward and the maximum expected future rewards. Through repeated application, the agent learns to approximate the optimal Q-values and can determine the best actions to maximize long-term rewards.

Source : Datacamp (https://www.datacamp.com/tutorial/bellman-equation-reinforcement-learning) Last accessed-20/01/2026

Proximal Policy Optimization (PPO) (1/2)

- **Algorithm Type:** Policy-based, On-policy, Model-free Reinforcement Learning.
- **Goal:** Learn a stochastic policy $\pi(a|s)$ that directly maps states to action probability distributions, maximizing expected cumulative reward.
- **No Environment Model:** Agent learns purely from experience with the environment.
- **Policy:** samples actions from a learned probability distribution over all possible actions
- **Neural Networks:** Uses actor-critic architecture → an actor network that outputs the policy $\pi(a|s)$ and a critic network that estimates the value function $V(s)$.
- **Clipped Objective Strategy:** Restricts the probability ratio to $[1-\varepsilon, 1+\varepsilon]$ (typically $\varepsilon=0.2$), preventing destabilizing large policy shifts.
- **Exploration:** Natural stochastic exploration through the policy's probability distribution via an entropy term.

Proximal Policy Optimization (PPO) (2/2)

Policy Objective Function

$$L^{PG}(\theta) = E_t[\log \pi_\theta(a_t|s_t) * A_t]$$

log probability of taking that action at that state

Advantage if A>0, this action is better than the other action possible at that state

PPO's Clipped surrogate objective function

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)\right]$$

The ratio function

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

Final PPO's Actor Critic Objective Function

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t\left[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)\right]$$

c1 and c2 are coefficients.

Add an entropy bonus **to ensure sufficient exploration**

Squared-error value loss: $(V_\theta(s_t) - V_t^{targ})^2$

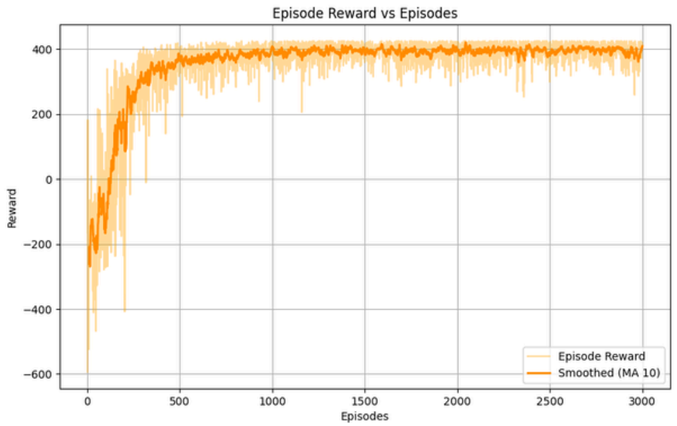
Source: https://huggingface.co/learn/deep-rl-course/unit8/clipped-surrogate-objective

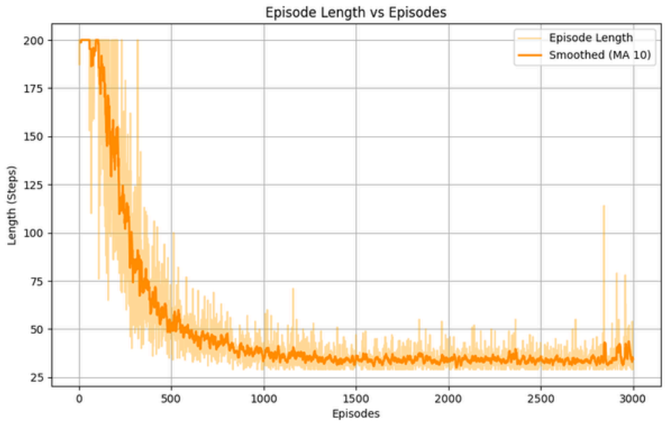
Training Details (1/2) - Q Learning

Training Configuration

Training Plots:

1. **Cumulative Reward**
2. **Loss - absolute Temporal Difference (TD) error**
3. **Mean Episode length**





Training Parameters

- No. of Episodes: 3000
- Learning Rate (α): 0.1
- Discount (γ): 0.99
- ε: 1.0 → 0.05 (decay 0.998)

Training Details (2/2) - PPO

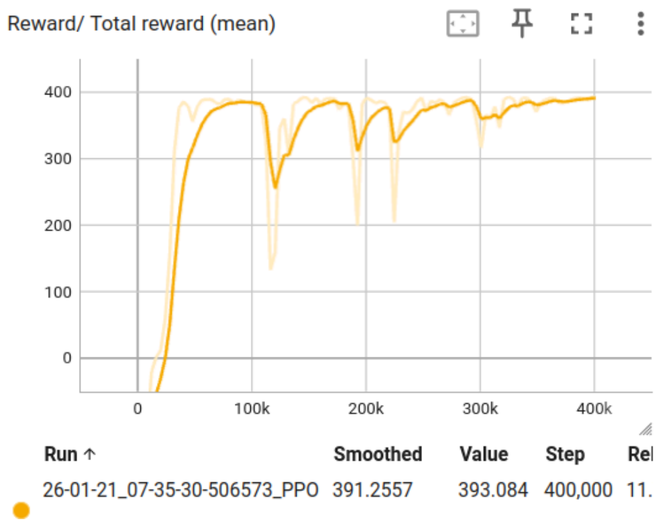
Training Configuration

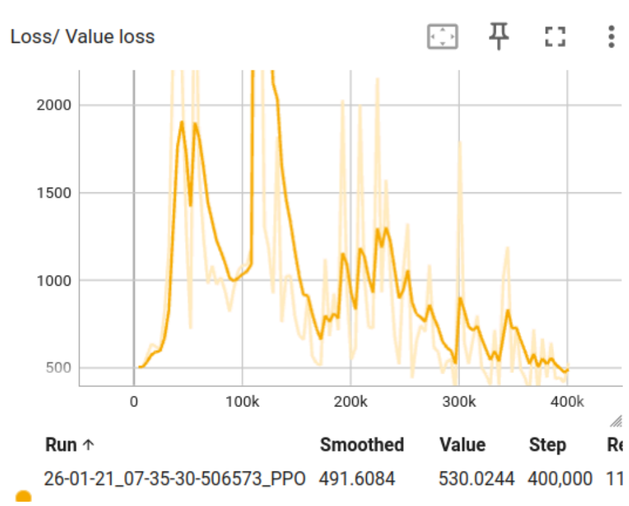
Training Plots:

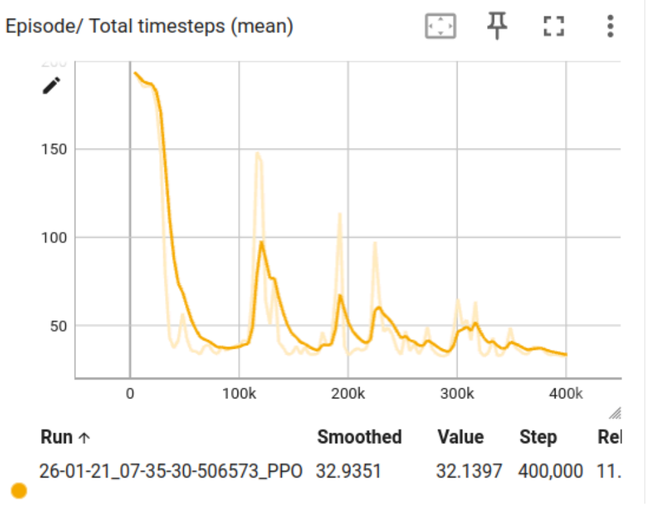
1. **Cumulative Reward**
2. **Value loss**
3. **Mean Episode length**

Hyper-parameters:

- Max Iterations: 400,000 (equivalent to 3000 episodes of Q learning) trained in 16 parallel envs.
- 3 hidden layers with 256 dimension size N/W for actor-critic N/W
  - Actor → 7 obs-> 3×256 -> 4 actions.
  - Critic → 7 obs → 3×256 → 1 value
- Learning Rate: 3.0×10^{-4} (Adam optimizer)
- PPO Clip: $\varepsilon = 0.2$ (prevents catastrophic failure)
- Entropy Bonus: 0.02 (minimal exploration)







Results and Evaluation

We validate the trained RL agents in the grid world for 100 episodes and evaluate their performance across four metrics.

Four metrics that track the performance of RL agent in reaching the consecutive goals while avoiding dynamic obstacles :-

1. Success Rate: Percentage of episodes where the agent successfully completed the consecutive goal sequence without exceeding the episode timeout.
2. Collision Rate: Percentage of episodes where the agent hit at least one moving obstacle during navigation.
3. Success-weighted Path Length (SPL): Ratio of optimal path length to actual path length
4. Average Steps to Reach Final Goal : Mean number of timesteps required to complete all 4 goals from start to finish

RL Agent	Success rate (100 episodes)	Collision rate (100 episodes)	Success weighted Path Length	Average steps to reach the final goal in a given episode
Q-Learning	100%	83%	0.91	32.23
PPO (Deep RL)	100%	65%	0.83	35.57

Conclusion

- Q-learning is suitable for discrete grids and effective for small scale problems, this can be seen through the better SPL it has compared to the PPO based agent.
- While both algorithms succeed at reaching all the goals as expected, PPO provides a policy with safer route with less collisions compared to Q-learning.
- Thus we can conclude for grid world with discrete and manageable action-space, Q learning is better suited than the PPO algorithm to derive an optimal RL policy agent.

Project Code on Github :- https://github.com/aaron1201/GridWorld-RL